

# Configuración de la red en Linux

## Sumario

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| 1. Introducción.....                                                          | 3  |
| 2. Detección de las tarjetas de red.....                                      | 3  |
| 3. Configuración a bajo nivel. Mediante comandos.....                         | 4  |
| 3.1. Configuración de las tarjetas a bajo nivel a través de ifconfig.....     | 8  |
| 3.2. Configuración de las tarjetas a bajo nivel a través de ip.....           | 9  |
| 4. Configuración de la red mediante networking y /etc/network/interfaces..... | 10 |
| 4.1. Activación/Desactivación de la tarjeta de red.....                       | 16 |
| 4.2. Pasos para modificar la configuración de la tarjeta de red.....          | 18 |
| 5. Configuración de la red mediante Netplan.....                              | 19 |
| 5.1. Fichero de configuración /etc/netplan/*.yaml.....                        | 20 |
| 5.2. Configuración de Netplan con NetworkManager.....                         | 21 |
| 5.3. Configuración de Netplan manualmente con networkd.....                   | 21 |
| 5.4. Comandos.....                                                            | 23 |
| 5.5. Ejemplos de configuraciones en Netplan.....                              | 23 |
| 6. Nuevas herramientas y comandos de red.....                                 | 26 |
| 6.1. Comando ip.....                                                          | 26 |
| 6.2. Mostrar dispositivos de red y su configuración.....                      | 27 |
| 6.3. Activar una interfaz de red.....                                         | 28 |
| 6.4. Desactivar una interfaz de red.....                                      | 28 |
| 6.5. Configurar una dirección IP en una interfaz.....                         | 29 |
| 6.6. Configurar varias direcciones IP en una interfaz.....                    | 29 |
| 6.7. Eliminar una dirección IP de una interfaz.....                           | 30 |
| 6.8. Añadir una interfaz virtual (alias).....                                 | 30 |
| 6.9. Activar/Desactivar el modo promiscuo de la tarjeta de red .....          | 30 |
| 6.10. Cambiar la MAC de una interfaz de red,.....                             | 31 |
| 6.11. Ver las entradas en la tabla ARP.....                                   | 31 |
| 6.12. Añadir una entrada en la tabla ARP.....                                 | 31 |
| 6.13. Quitar una entrada de ARP.....                                          | 31 |
| 6.14. Comandos «ip route» para configurar rutas.....                          | 31 |
| 6.15. Listado de comandos básicos de ss.....                                  | 32 |
| a) Ver estadísticas de sockets a la escucha.....                              | 33 |
| b) Mostrar conexiones ssh establecidas.....                                   | 33 |
| c) Mostrar conexiones HTTP establecidas.....                                  | 34 |
| d) Hacer coincidir direcciones remotas y puertos.....                         | 34 |
| e) Mostrar el número y el tipo de conexiones activas.....                     | 34 |
| f) Listar todos los puertos a la escucha junto con el PID del programa.....   | 34 |
| g) Mostrar un resumen del estado de los sockets.....                          | 35 |
| 6.16. Hacer persistentes todos los cambios en IPs y rutas.....                | 35 |
| 7. El nombre del equipo - /etc/hostname.....                                  | 35 |
| 8. Hacer un equipo que sea enrutador.....                                     | 35 |
| 10. Configuración NAT.....                                                    | 37 |
| 10.1. activación del bit de forward.....                                      | 37 |

|                                                        |    |
|--------------------------------------------------------|----|
| 10.1. Postrouting.....                                 | 37 |
| 10.2. Source nat (estático) con iptables.....          | 38 |
| 10.3. Prerouting.....                                  | 39 |
| 11. Guardar reglas iptables.....                       | 42 |
| 11.1. Utilización del paquete iptables-persistent..... | 42 |

## 1. Introducción

En linux tenemos diferentes sistemas para configurar la red:

Modo gráfico.

- Network Manager.

Modo consola

- Configuración a bajo nivel. Mediante comandos.
- Networking y fichero /etc/interfaces.
- Netplan.

## 2. Detección de las tarjetas de red

Antes de configurar las tarjetas de red, estas deben haber sido detectadas por el kernel y haberse cargado correctamente los módulos correspondientes para su manejo.

Podemos consultar con el comando `dmesg`, los mensajes que el kernel mostró al arrancar el sistema, pero como solo nos interesan los relacionados con las tarjetas de red, en este caso de tipo ethernet, filtramos solo las líneas que contenga el texto `eth`:

```
# dmesg | grep eth
[ 1.703665] e1000 0000:00:03.0: eth0: (PCI:33MHz:32-bit) 08:00:27:93:ff:13
[ 1.703671] e1000 0000:00:03.0: eth0: Intel(R) PRO/1000 Network Connection
[ 11.684947] e1000: eth0 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: RX
[ 11.686082] ADDRCONF(NETDEV_UP): eth0: link is not ready
[ 11.686101] ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
[ 22.208445] eth0: no IPv6 routers present
[19228.344206] e1000: eth0 NIC Link is Down
[19234.356477] e1000: eth0 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: RX
[29238.719257] ADDRCONF(NETDEV_UP): eth0: link is not ready
[29238.721456] e1000: eth0 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: RX
[29238.721839] ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
[29249.568108] eth0: no IPv6 routers present
```

Como podemos observar, no hay ningún tipo de error, por lo que el kernel detecta correctamente las tarjetas de red, en este caso, solo hay una, eth0.

Podemos comprobar con el comando `lsmod`, que el módulo del kernel que maneja la tarjeta de red (e1000) está cargado. Este comando muestra todos los módulos cargados por el kernel, si conocemos el nombre del módulo podríamos hacer un filtro como en el comando anterior:

```
# lsmod
Module                Size  Used by
vboxsf                32542  1
bnep                  17288  2
libata                125014  4 libahci,ahci,ata_piix,ata_generic
scsi_mod              135037  4 libata,sd_mod,sr_mod,sg
e1000                 80417  0
usb_common            12338  1 usbcore
Anexo:
```

Los mensajes que el kernel va generando en el arranque del sistema, se guardan en el fichero `/var/log/dmesg`, y el comando `dmesg` muestra solo una parte de estos mensajes, los cuales aún se encuentran en la RAM (ring buffer).

Para una tarjeta wifi

```
$ dmesg | grep wlo
[ 3.474445] iwlwifi 0000:00:14.3 wlo1: renamed from wlan0
```

Se muestra el dispositivo de la tarjeta wifi

```
$ lsmod | grep iwlwifi
iwlwifi              372736  1 iwlvm
cfg80211             888832  3 iwlvm,iwlwifi,mac80211
```

Se comprueba que esta cargado el módulo del kernel que maneja la tarjeta de red.

### 3. Configuración a bajo nivel. Mediante comandos

Es posible hacer una configuración de la red con los comandos ifconfig, route e ip, es lo que se denomina configuración a bajo nivel, pero el **inconveniente de este tipo configuraciones es que no se mantienen al reiniciar el sistema, a menos que se ejecuten desde un script en el arranque.**

**Los comandos ifconfig y route (paquete net-tools) están hoy en día en desuso y han sido sustituidos por el comando ip (paquete iproute).**

Usaremos estos comandos para obtener información de la red y no para configurarla salvo en situaciones especiales como por ejemplo en el caso del mapeo lógico de un dispositivo de red.

Para ver la configuración de las interfaces de red activas ejecutamos:

```
# ifconfig
eth0    Link encap:Ethernet  HWaddr 08:00:27:28:42:d2
        inet addr:192.168.1.100  Bcast:192.168.1.255  Mask:255.255.255.0
        inet6 addr: fe80::a00:27ff:fe28:42d2/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:2933 errors:0 dropped:0 overruns:0 frame:0
        TX packets:837 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:657635 (642.2 KiB)  TX bytes:126005 (123.0 KiB)

lo      Link encap:Local Loopback
        inet addr:127.0.0.1  Mask:255.0.0.0
        inet6 addr: ::1/128 Scope:Host
        UP LOOPBACK RUNNING  MTU:16436  Metric:1
        RX packets:215 errors:0 dropped:0 overruns:0 frame:0
        TX packets:215 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:0
        RX bytes:14304 (13.9 KiB)  TX bytes:14304 (13.9 KiB)
```

Se podrían ver todas las interfaces de red que tiene el sistema, tanto las activas (llevan la palabra UP) como las inactivas (no llevan la palabra UP), para lo cual, ejecutamos:

```
# ifconfig -a
eth0    Link encap:Ethernet  HWaddr 08:00:27:28:42:d2
```

```

inet addr:192.168.1.100 Bcast:192.168.1.255 Mask:255.255.255.0
inet6 addr: fe80::a00:27ff:fe28:42d2/64 Scope:Link
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:44 errors:0 dropped:0 overruns:0 frame:0
TX packets:83 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:13149 (12.8 KiB) TX bytes:10400 (10.1 KiB)

eth1    Link encap:Ethernet HWaddr 08:00:27:37:d8:06
        BROADCAST MULTICAST MTU:1500 Metric:1
        RX packets:0 errors:0 dropped:0 overruns:0 frame:0
        TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

lo      Link encap:Local Loopback
        inet addr:127.0.0.1 Mask:255.0.0.0
        inet6 addr: ::1/128 Scope:Host
        UP LOOPBACK RUNNING MTU:16436 Metric:1
        RX packets:12 errors:0 dropped:0 overruns:0 frame:0
        TX packets:12 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:0
        RX bytes:720 (720.0 B) TX bytes:720 (720.0 B)

```

Podemos ver una estadística de la información enviada y recibida por las interfaces activas o todas:

```

# ifconfig -s

```

| Iface | MTU   | Met | RX-OK | RX-ERR | RX-DRP | RX-OVR | TX-OK | TX-ERR | TX-DRP | TX-OVR |
|-------|-------|-----|-------|--------|--------|--------|-------|--------|--------|--------|
| eth0  | 1500  | 0   | 53    | 0      | 0      | 0      | 83    | 0      | 0      | 0      |
| lo    | 16436 | 0   | 12    | 0      | 0      | 0      | 12    | 0      | 0      | 0      |

```

Flg
eth0 BMRU
lo LRU

```

Podemos ver los datos de una interfaz concreta:

```
# ifconfig eth0
```

```
eth0    Link encap:Ethernet  HWaddr 08:00:27:28:42:d2
        inet addr:192.168.1.100  Bcast:192.168.1.255  Mask:255.255.255.0
        inet6 addr: fe80::a00:27ff:fe28:42d2/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:62 errors:0 dropped:0 overruns:0 frame:0
        TX packets:85 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:16187 (15.8 KiB)  TX bytes:10582 (10.3 KiB)
```

Para mostrar la tabla arp usamos el comando arp:

```
# arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
192.168.1.1      ether   38:72:c0:b5:1f:23  C           eth0
# arp -a
? (192.168.1.1) at 38:72:c0:b5:1f:23 [ether] on eth0
```

La opción -n hace que se muestren todos los datos de forma numérica y no se intente hacer traducción de IP a nombre; esto hace, generalmente, que el comando funcione más rápido que obviando la opción.

La opción -a muestra la misma información con otro tipo de formato de salida.

Para mostrar las tablas de enrutamiento usamos el comando route:

```
# route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         192.168.1.1    0.0.0.0         UG    0     0     0 eth0
192.168.1.0     0.0.0.0        255.255.255.0   U     0     0     0 eth0
La opción -n tiene la misma función que en el comando arp.
```

También se puede utilizar la orden ip para obtener la información de la red. Así, para ver la lista de tarjetas que hay instaladas en el sistema, con su dirección MAC y si están activas o no, ejecutamos:

```
# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state UNKNOWN mode
DEFAULT
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
mode DEFAULT qlen 1000
    link/ether 08:00:27:28:42:d2 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT
qlen 1000
    link/ether 08:00:27:37:d8:06 brd ff:ff:ff:ff:ff:ff
```

El parámetro show puede suprimirse, en la orden anterior y en las sucesivas, se asume por defecto.

Para ver las direcciones IP de las tarjetas:

# ip address show / es lo mismo que ip a

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
qlen 1000
    link/ether 08:00:27:28:42:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
    inet6 fe80::a00:27ff:fe28:42d2/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN qlen 1000
    link/ether 08:00:27:37:d8:06 brd ff:ff:ff:ff:ff:ff
```

### 3.1. Configuración de las tarjetas a bajo nivel a través de ifconfig

En este anexo se va a resumir cómo se pueden utilizar los comandos `ifconfig` e `ip` para configurar las tarjetas de red.

**Este método se utiliza cuando no es posible mediante otros modos.**

La sintaxis resumida de `ifconfig` para asignar una IP a una tarjeta de red es la siguiente:

```
ifconfig interfaz dirección-ip netmask máscara [up]
```

Si se especifica `up` al final del comando, entonces la tarjeta se levanta a la vez que se le asigna la dirección IP. Por ejemplo:

```
# ifconfig eth0 192.168.0.2 netmask 255.255.255.0 up
```

Con los siguientes comandos se activa o desactiva la tarjeta de red:

```
ifconfig interfaz { up | down }
```

```
# ifconfig eth0 down  
# ifconfig eth0 up
```

### 3.2. Configuración de las tarjetas a bajo nivel a través de `ip`.

El comando `ip` realiza las mismas funciones que los comandos `ifconfig`, `route` y `arp` a la vez. Para poner una dirección IP, la sintaxis resumida sería:

```
ip address add dirección-ip/prefijo dev interfaz
```

```
# ip address add 192.168.0.2/24 dev eth0
```

Si queremos cambiar de IP es necesario borrar previamente la dirección anterior para no asignar dos direcciones. La sintaxis para borrar una dirección IP es:

```
ip address del dirección-ip/prefijo dev interfaz
```

```
# ip address del 192.168.0.2/24 dev eth0
```

Por último falta ver la sintaxis del comando ip para activar y desactivar la tarjeta de red:

```
ip link set interfaz { up | down }
```

```
# ip link set eth0 down
```

```
# ip link set eth0 up
```

#### **4. Configuración de la red mediante networking y /etc/network/interfaces**

El fichero `/etc/network/interfaces` se utiliza para definir los nombres lógicos de las interfaces de redes, a los que se les asociará una configuración determinada. El comando `ifup` es el encargado de relacionar una interfaz física con una interfaz lógica.

El contenido de este fichero está formado por líneas de diferentes tipos:

- Líneas `auto` : sirven para especificar las interfaces físicas que se activarán cuando se ejecute el comando `ifup -a`, el cual se ejecuta automáticamente al arrancar el sistema, por lo que especifica las tarjetas que automáticamente se activarán desde el inicio. Se pueden poner todas las líneas de este tipo que se deseen y en cada línea pueden ir más de una tarjeta física, simplemente separándolas con un espacio.

```
auto eth0 eth2 eth3
auto eth1
```

En las líneas anteriores la interfaz física `eth0` se asocia con configuración que posea la interfaz lógica `eth0`, la `eth1` con la `eth1`, etc.

- Líneas `allow-auto` : son equivalentes a las líneas `auto`.
- Líneas `allow-hotplug` : son parecidas a las líneas `auto`. Las interfaces físicas que aparecen en estas líneas se activan cuando se producen eventos `hotplug` en las interfaces de red, como la detección de la tarjeta por parte del kernel, la conexión del cable de red, etc. Cuando se producen estos eventos, el sistema ejecuta el comando `ifup` asociado a la tarjeta de red involucrada. Igualmente se asocian con una configuración lógica del mismo nombre.

```
allow-hotplug eth0 eth2 eth3
allow-hotplug eth1
```

- Líneas `iface` : sirven para definir nombres lógicos de interfaces de red junto con su configuración particular.
- Líneas `mapping` : sirven para definir cómo se asociará automáticamente una interfaz física a una lógica de entre un grupo de estas.
- Líneas `#` : sirven para especificar comentarios.
- Líneas que acaban en `\` : sirven para extender la configuración de la línea anterior.

Ejemplos:

```
$ cat /etc/network/interfaces
auto lo
iface lo inet loopback
auto eth0
iface eth0 inet dhcp
```

```
$ cat /etc/network/interfaces
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 192.168.11.100
    netmask 255.255.255.0
    gateway 192.168.11.1
```

```
$ cat /etc/network/interfaces
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp

auto eth1
iface eth1 inet static
    address 192.168.11.100
    netmask 255.255.255.0
    gateway 192.168.11.1
```

Las líneas iface tienen la siguiente sintaxis:

```
iface <config_name> <address_family> <method_name>
```

```
<option1> <value1>  
<option2> <value2>
```

donde <config\_name> será un nombre lógico de interfaz de red, <address\_family> normalmente contendrá el valor inet para especificar una configuración IPv4, pero también puede ser inet6 (IPv6) o ipx (IPX). El valor de <method\_name> indica el método de configuración de la interfaz y depende del valor de <address\_family>, para el caso de la familia inet los métodos son:

- loopback: usado solo para la interfaz lo (bucle local).
- static: usado para asignar una dirección ip fija.
- dhcp: usado para asignar una dirección dinámica.

Las opciones finales (parejas opción/valor) van a depender de la familia y del método elegido.

Las dos primeras líneas del fichero /etc/network/interfaces deben ser:

```
auto lo  
iface lo inet loopback
```

las cuales activan la interfaz lo (bucle local), que usaremos en tareas de diagnóstico de conectividad y validez de los protocolos de comunicación.

El método static, usado para asignar una ip fija a una interfaz de red, tiene las siguientes opciones:

| Opción                      | Ejemplo                  | Significado                                            |
|-----------------------------|--------------------------|--------------------------------------------------------|
| <i>address dirección-IP</i> | address<br>192.168.10.15 | Establece la dirección IP. (obligatorio)               |
| <i>netmask máscara</i>      | netmask                  | Establece la máscara de la dirección IP. (obligatorio) |

|                                              |                                          |                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                              | 255.255.255.0                            |                                                                                                                                                                                                                                                                                                                  |
| <i>gateway dirección-IP</i>                  | gateway<br>192.168.10.1                  | Establece la puerta de enlace. En un equipo sólo puede haber una puerta de enlace, o lo que es lo mismo, sólo puede haber una regla por defecto de enrutamiento. Esta opción se asociará a la línea <i>iface</i> cuya ip se corresponde con la ip de salida de la regla por defecto en la tabla de enrutamiento. |
| <i>network dirección-IP-de-red</i>           | network<br>192.168.10.0                  | Establece la dirección de la red a la que pertenece la dirección IP.                                                                                                                                                                                                                                             |
| <i>broadcast dirección-IP-de-broadcast</i>   | broadcast<br>192.168.10.255              | Establece la dirección de broadcast de la red.                                                                                                                                                                                                                                                                   |
| <i>hwaddress tipo-interfaz dirección-MAC</i> | hwaddress ether<br>00:01:04:1b:2C:1<br>F | Establece la dirección MAC de la tarjeta. Para las tarjetas ethernet se debe poner la palabra <i>ether</i> como tipo de la interfaz.                                                                                                                                                                             |
| <i>pre-up comando</i>                        | pre-up echo...                           | Ejecuta <i>comando</i> antes de que la tarjeta se configure con el comando <i>ifup</i> . Si el comando falla, <i>ifup</i> aborta, imprime un mensaje de error y la tarjeta no se marca como configurada.                                                                                                         |
| <i>up comando</i>                            | up sleep 2 ; route<br>add...             | Ejecuta <i>comando</i> durante la configuración de la tarjeta con el comando <i>ifup</i> .                                                                                                                                                                                                                       |
| <i>post-up comando</i>                       | post-up route<br>add...                  | Ejecuta <i>comando</i> después de que la tarjeta se configure con el comando <i>ifup</i> . Si el comando falla, <i>ifup</i> aborta, imprime un mensaje de error y la tarjeta no se marca como configurada aunque haya sido configurada.                                                                          |
| <i>pre-down comando</i>                      | pre-down route<br>del...                 | Ejecuta <i>comando</i> antes de que la tarjeta se desconfigure con el comando <i>ifdown</i> . Si el comando falla, <i>ifdown</i> aborta, imprime un mensaje de error y la tarjeta se marca como desconfigurada aunque no                                                                                         |

|                          |                                             |                                                                                                                                                                                                                      |
|--------------------------|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          |                                             | haya sido desconfigurada.                                                                                                                                                                                            |
| <i>down comando</i>      | <code>down sleep 2 ;</code><br>route del... | Ejecuta <i>comando</i> durante la desconfiguración de la tarjeta con el comando <i>ifdown</i> .                                                                                                                      |
| <i>post-down comando</i> | post-down echo...                           | Ejecuta <i>comando</i> después de que la tarjeta se desconfigure con el comando <i>ifdown</i> . Si el comando falla, <i>ifdown</i> aborta, imprime un mensaje de error y la tarjeta no se marca como desconfigurada. |

Las opciones pre-up, up, post-up, pre-down, down y post-down tienen las siguientes características:

- Son opcionales.
- Son opciones válidas para cualquier familia y método.
- Pueden repetirse cuantas veces queramos.
- Se ejecutan cada una en su momento, y dentro del mismo tipo, en el orden de aparición en el fichero.
- Se puede asegurar que siempre se ejecute correctamente el comando, ya que a este le podemos añadir el sufijo "|| true" creando una lista condicional OR (true es un comando que no hace nada y tiene como código de salida el valor 0, false es el comando contrario, es decir, tiene como código de salida el valor 1, y tampoco hace nada).

Además de las líneas up, down, etc. del fichero /etc/network/interfaces, existe otra posibilidad de ejecutar comandos cuando una tarjeta de red se está activando o desactivando; esta consiste en introducir scripts dentro de cualquiera de los directorios:

- /etc/network/if-down.d
- /etc/network/if-post-down.d
- /etc/network/if-pre-up.d
- /etc/network/if-up.d

que equivalen a los momentos down, post-down, pre-up y up, respectivamente.

El método dhcp, usado para asignar una ip dinámicamente a la interfaz de red, tiene las siguientes opciones con el mismo significado anterior:

- hwaddress
- pre-up, up, post-up
- pre-down, down, post-down

Para obtener más información sobre el fichero /etc/network/interfaces podemos acceder al manual del sistema:

```
# man interfaces
```

#### 4.1. Activación/Desactivación de la tarjeta de red

A continuación se describen los tres comandos relacionados con el demonio networking y su fichero de configuración /etc/network/interfaces, para activar y desactivar interfaces de red.

##### ifup

Sirve para activar una interfaz de red.

La sintaxis del comando ifup es la siguiente:

```
ifup [opción...] interfaz-física[=interfaz-lógica]...
```

Si ejecutamos las instrucción:

```
# ifup eth1
```

se activa la interfaz física eth1 con la configuración de la interfaz lógica del mismo nombre, es decir, eth1.

Si quisiéramos activar la interfaz eth2 con la configuración de la interfaz lógica de nombre oficina, ejecutaríamos:

```
# ifup eth2=oficina
```

También es posible levantar todas las tarjetas que se hayan especificado en las líneas auto de /etc/network/interfaces:

```
# ifup -a
```

## ifdown

La orden ifdown es la encargada de desconfigurar las interfaces de red. Su sintaxis es la siguiente:

```
ifdown [opción...] interfaz-física...
```

Por ejemplo, para desactivar la interfaz física eth0 ejecutaríamos:

```
# ifdown eth0
```

Todas las interfaces físicas activas pueden desconfigurarse ejecutando:

```
# ifdown -a
```

## ifquery

El comando ifquery permite consultar las opciones de configuración de una interfaz física que esté activa o de una interfaz lógica del fichero /etc/network/interfaces, la cual no es necesario que esté asociada a una interfaz física.

```
# ifquery eth0
address: 192.168.1.100
netmask: 255.255.255.0
gateway: 192.168.1.1
network: 192.168.1.0
broadcast: 192.168.1.255
```

## 4.2. Pasos para modificar la configuración de la tarjeta de red

Para modificar la configuración de una tarjeta de red con el demonio networking, hay que seguir en orden los siguientes pasos:

1. Desactivar la interfaz física de red ejecutando `ifdown`. Se desconfigurará en base a los parámetros que haya en el fichero `/etc/network/interfaces` para su interfaz lógica asociada.
2. Editar el fichero `/etc/network/interfaces`.
3. Activar la tarjeta física de red ejecutando `ifup`. Se configurará en base a los parámetros que haya en el fichero `/etc/network/interfaces` para la interfaz lógica asociada.

## 5. Configuración de la red mediante Netplan

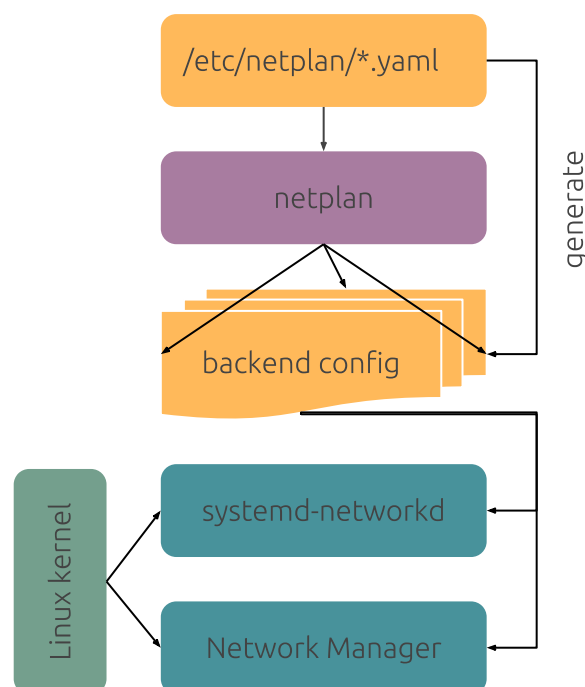
Netplan es una nueva utilidad de configuración de red, de línea de comandos para administrar y configurar los ajustes de red, de forma fácil. Permite configurar al usuario una interfaz de red utilizando el formato YAML. Funciona junto a los daemon de red como NetworkManager y systemd-networkd, como interfaces para el kernel.

Netplan se encarga de leer la configuración indicada en los ficheros de la carpeta `/etc/netplan/*.yaml` y puede almacenar las configuraciones para todas las interfaces de red en estos archivos.

Durante el inicio inicial, Netplan genera archivos de configuración específicos del back-end para transferir /runel control de los dispositivos a un demonio de red en particular.

Netplan admite dos formas de configurar la máquina

- NetworkManager
- Systemd-networkd



Básicamente NetworkManager la utilizaremos para entornos de escritorio gráficos, y Systemd-networkd para entornos servidor o cuando quieres configurar la red de forma manual.

**Cuando se instala netplan deja de generarse el fichero /etc/network/interfaces**

### 5.1. Fichero de configuración /etc/netplan/\*.yaml

El fichero de configuración de netplan esta en formato yaml. YAML solo admite espacios y tiene la capacidad de distinguir entre mayúsculas y minúsculas.

Reglas

- Cada par etiqueta/valor debe estar indentado por tres espacios a partir del comienzo de la línea, siguiendo el ejemplo anterior quedaría algo así:
- Yaml no admite tabulaciones.
- Hay que dejar espacio entre parejas de valores.

Por ejemplo:

```
dhcp6:no
addresses:[10.10.0.11/24]
```

No sería válido, lo correcto es:

```
dhcp6: no
addresses: [10.10.0.11/24]
```

Para ayudarnos con la sintaxis del fichero tenemos el comando:

```
sudo netplan --debug generate
```

```
root@ubuntu:~# netplan --debug generate
DEBUG:command generate: running ['/lib/netplan/generate']
** (generate:1728): DEBUG: 02:17:41.461: Processing input file /etc/netplan/01-
netcfg.yaml..
** (generate:1728): DEBUG: 02:17:41.461: starting new processing pass
```

```
** (generate:1728): DEBUG: 02:17:41.461: ens33: setting default backend to 1
** (generate:1728): DEBUG: 02:17:41.461: Generating output files..
** (generate:1728): DEBUG: 02:17:41.462: NetworkManager: definition ens33 is not
for us (backend 1) netplan tenemos
```

Se necesita permisos de administrador.

## 5.2. Configuración de Netplan con NetworkManager.

La forma de decirle a netplan que el gestor de la red va a ser el administrador gráfico es escribiendo el en fichero `/etc/netplan/*.yaml`

```
network:
  version: 2
  renderer: NetworkManager
```

## 5.3. Configuración de Netplan manualmente con networkd

Si queremos empezar a gestionar nosotros mismos la configuración de red del equipo, deberás cambiar «NetworkManager» por «networkd» y después decidir si la asignación de la ip's van a ser mediante dhcp o mediante asignación estática.

### Configuración mediante dhcp.

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp3s0:
      dhcp4: true
```

### Configuración mediante asignación estática.

```
network:
  version: 2
```

```
renderer: networkd
ethernets:
  enp3s0:
    addresses:
      - 10.10.10.2/24
    nameservers:
      search: [mydomain, otherdomain]
      addresses: [10.10.10.1, 1.1.1.1]
    routes:
      - to: default
        via: 10.10.10.1
```

```
network:
  version: 2
  renderer: networkd
ethernets:
  ens33:
    addresses: [192.168.1.50/24]
    gateway4: 192.168.1.1
    nameservers:
      search: [mytcpip.local]
      addresses: [192.168.1.1]
```

```
network:
  version: 2
  renderer: networkd
ethernets:
  ens3:
    dhcp4: yes
  eno2:
    dhcp4: no
    dhcp6: no
    addresses: [10.10.0.11/24]
```

```
gateway4: 10.10.0.1
routes:
  - to: 192.168.1.0/24
    10.10.0.1
  - to: 192.168.10.0/24
    via: 10.10.0.1
```

## 5.4. Comandos

Netplan utiliza un conjunto de subcomandos para impulsar su comportamiento:

- **netplan generate** : se utiliza /etc/netplan para generar la configuración necesaria para los renderizadores.
- **netplan apply** : aplica toda la configuración para los renderizadores, reiniciándolos según sea necesario.
- **netplan try** : Aplicar la configuración y esperar la confirmación del usuario; retroceder si la red está rota o no se da confirmación.

## 5.5. Ejemplos de configuraciones en Netplan

### Varias tarjetas configuradas mediante dhcp

Muchos sistemas ahora incluyen más de una interfaz de red. Los servidores normalmente necesitarán conectarse a múltiples redes y pueden requerir que el tráfico a Internet pase por una interfaz específica a pesar de que todos ellos proporcionen una puerta de enlace válida.

Se puede lograr el enrutamiento exacto deseado a través de DHCP especificando una métrica para las rutas recuperadas a través de DHCP, lo que garantizará que se prefieran algunas rutas sobre otras. En este ejemplo, se prefiere 'enp0s3' a 'enp0s8', ya que tiene una métrica de ruta más baja:

```
network:
```

```
version: 2
ethernets:
  enp0s3:
    dhcp4: yes
    dhcp4-overrides:
      route-metric: 100
  enp0s8::
    dhcp4: yes
    dhcp4-overrides:
      route-metric: 200
```

## Uso de múltiples direcciones en una sola interfaz

La clave de direcciones puede tomar una lista de direcciones para asignar a una interfaz:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp3s0:
      addresses:
        - 10.100.1.37/24
        - 10.100.1.38/24:
            label: enp3s0:0
        - 10.100.1.39/24:
            label: enp3s0:some-label
      routes:
        - to: default
          via: 10.100.1.1
```

## Uso de varias direcciones con varias puertas de enlace

Similar al ejemplo anterior, las interfaces con múltiples direcciones se pueden configurar con múltiples puertas de enlace.

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp3s0:
      addresses:
        - 10.0.0.10/24
        - 11.0.0.11/24
      routes:
        - to: default
          via: 10.0.0.1
          metric: 200
        - to: default
          via: 11.0.0.1
          metric: 300
```

Configuramos rutas individuales por defecto (o 0.0.0.0/0) utilizando la dirección de la puerta de enlace para la subred. El metricvalor debe ajustarse para que el enrutamiento ocurra como se esperaba.

Se puede usar DHCP para recibir una de las direcciones IP para la interfaz. En este caso, la ruta predeterminada para esa dirección se configurará automáticamente con un metricvalor de 100.

Mas información sobre Netplan en:

<https://netplan.io/>

## 6. Nuevas herramientas y comandos de red.

Linux viene con un conjunto de herramientas de red para administrar infraestructuras de red tanto a gran como a pequeña escala. Por mucho tiempo, net-tools eran las herramientas de red predeterminadas preferidas, pero ahora han sido reemplazadas por iproute2 herramientas.

El propósito de iproute2 es reemplazar el conjunto de herramientas que componen las net-tools y pasar a ser él quien se encargue de configurar las interfaces de red, la tabla de rutas y gestionar la tabla ARP.

### 6.1. Comando ip

La herramienta principal de iproute2 es el comando «ip», con el que podremos ver y configurar direcciones IP, ver y configurar tablas de enrutamiento, ver y configurar túneles IP, y también ver y configurar la interfaz física.

La sintaxis que debemos utilizar con este comando es la siguiente:

```
ip [ OPCIONES ] OBJETO [ COMANDO ]
```

Los «OBJETOS» que tenemos en ip son los siguientes, y son los que podremos ver y configurar los enlaces físicos, direcciones ip, rutas y más. Es muy importante que todas las órdenes estén tal y como os las ponemos, todo en minúsculas, de lo contrario, devolverá error. A continuación, tenéis los principales:

- link: nos sirve para configurar las interfaces de red físicas o lógicas, por ejemplo, para ver el estado de todas las interfaces de red.
- address: permite ver y configurar las direcciones IPv4 y IPv6 asociadas a las diferentes interfaces de red. Cada interfaz debe tener al menos, una dirección IP configurada.
- addlabel: permite añadir una etiqueta
- neighbour: permite ver los enlaces de vecindad, es decir, se puede ver la tabla ARP del sistema operativo.
- rule: permite ver y configurar políticas de enrutado y cambiarlas, esto se utiliza sobre todo cuando vas a configurar varias tablas de enrutamiento.

- route: permite ver y configurar las tablas de enrutamiento, tanto de la tabla de enrutamiento principal, como de las «secundarias» que configures.
- tunnel: permite ver los túneles IP y también configurarlos.
- maddr: permite ver y configurar las direcciones multienlace.
- mroute: permite ver y configurar la tabla de enrutamiento multicast.
- mrule: permite ver y configurar políticas de enrutamiento de direcciones multicast.
- monitor: permite monitorizar el estado de las tarjetas de red de manera continua, también direcciones IP y rutas.
- ntable: gestiona el caché de neighbour (ARP)
- tuntap: gestiona las interfaces TUN/TAP, orientado a las VPN como OpenVPN o WireGuard.
- maddress: configuración de las direcciones multicast
- xfrm: gestiona las políticas IPsec.
- netns: administrar espacios de nombres de red
- l2tp: configuración de L2TP
- tcp\_metrics: gestiona métricas TCP.
- token: gestiona los identificadores con token de las interfaces.

## 6.2. Mostrar dispositivos de red y su configuración

**Comando antiguo.** Ifconfig

**Comando actual** ip addr show / ip link show

Con el comando ip, tenemos dos comandos parecidos, uno de ellos nos mostrará toda la información a nivel de direccionamiento IP (ip addr show), y el otro nos mostrará el estado de la interfaz física (ip link show):

Estos dos comandos son muy importantes, porque podremos ver toda la información de direccionamiento IP, máscara de subred y muchos otros datos importantes para la red local, además, también podremos ver cómo tenemos configurada la interfaz física, el MTU que tenemos configurado e incluso si está o no habilitada esta interfaz de red.

```
$ sudo ifconfig -a
```

Con iproute2, el siguiente comando nos da una salida similar a ifconfig.

```
$ sudo ip addr list
```

o también:

```
$ sudo ip addr show
```

Con el siguiente, vemos la información en capa 2 (data link layer) del Modelo OSI:

```
$ sudo ip link show
```

### 6.3. Activar una interfaz de red

**Comando antiguo.** ifconfig ens33 up

**Comando actual .** ip link set ens33 up

Si queremos activar una interfaz de red (levantarla) porque está desactivada, el comando es muy similar al del antiguo «ifconfig», pero ahora tendremos que hacer uso de «ip link» para configurar esta característica. El funcionamiento es realmente sencillo, en este caso, los comandos son muy parecidos.

```
$ sudo ifconfig eth0 up
```

```
$ sudo ip link set eth0 up
```

### 6.4. Desactivar una interfaz de red

**Comando antiguo.** ifconfig ens33 down

**Comando actual.** ip link set ens33 down

Si queremos deshabilitar o desactivar una interfaz de red, los comandos con `ifconfig` y con `ip link` son casi iguales, tal y como ocurría anteriormente para activar la red.

```
$ sudo ifconfig eth0 down
$ sudo ip link set eth0 down
```

## 6.5. Configurar una dirección IP en una interfaz

**Comando antiguo.** `ifconfig ens33 192.168.1.1/24`

**Comando actual.** `ip addr add 192.168.1.1/24 dev ens33`

Configurar una dirección IP en una interfaz es realmente sencillo, tanto con `ifconfig` como también con el comando «`ip`», y es que la sintaxis es muy parecida a la sintaxis utilizada en los routers Cisco con el típico «`ip addr`» que todos hemos utilizado alguna vez.

Con este comando estaremos indicando que queremos poner una dirección IP y una máscara de subred específica a una interfaz que nosotros hemos definido.

```
Con ifconfig:
$ sudo ifconfig eth0 192.168.1.1/24
o también:

$ sudo ifconfig eth0 192.168.1.1 netmask 255.255.255.0
Con iproute2:

$ sudo ip address add 192.168.1.1/24 dev eth0
```

## 6.6. Configurar varias direcciones IP en una interfaz

`iproute2` permite asignar múltiples direcciones IPv4 a una interfaz, cosa que no es posible hacer mediante `ifconfig`.

```
$ sudo ip addr add 10.0.0.1/24 broadcast 10.0.0.255 dev eth1
$ sudo ip addr add 10.0.0.2/24 broadcast 10.0.0.255 dev eth1
$ sudo ip addr add 10.0.0.3/24 broadcast 10.0.0.255 dev eth1
```

## 6.7. Eliminar una dirección IP de una interfaz.

**Comando antiguo.** No soportada. Había que poner 0.0.0.0 a la interfaz.

**Comando actual.** `ip addr del 192.168.1.1 dev ens33`

En el caso de `ip`, es tan fácil como sustituir el «add» para configurar una IP, por «del» para eliminarla (delete).

```
$ sudo ifconfig eth0 0.0.0.0
$ sudo ip addr del 192.168.1.1/24 dev eth0
```

## 6.8. Añadir una interfaz virtual (alias)

**Comando antiguo.** `ifconfig ens33:1 10.0.0.1/8`

**Comando actual.** `ip addr add 10.0.0.1/8 dev ens33 label ens33:redeszone`

En todas las interfaz físicas tenemos la posibilidad de configurar interfaces virtuales para otro tipo de usocs, esto también se le conoce como «alias».

Con `ip`, simplemente tenemos que poner la IP de siempre, pero con «label ens33:1» o el alias que queramos.

Mientras que con «ifconfig» no es necesario poner ningún tipo de «label», en el caso de «ip» sí tendremos que colocarlo a modo de etiqueta, además, la parte positiva de este label es que podremos nombrar la interfaz de red virtual recién creada como nosotros queramos, no es necesario que siga la sintaxis ens33:1 o lo que sea, podremos poner un nombre.

Para ejecutar la ayuda de «ip addr» y ver todos los comandos que podemos ejecutar, simplemente basta con poner:

```
ip addr help
```

Debemos recordar que la ayuda de «ip» es realmente útil y nos permitirá saber qué comandos se permiten ejecutar.

## 6.9. Activar/Desactivar el modo promiscuo de la tarjeta de red .

```
$ sudo ip link set dev eth0 promisc on
```

```
$ sudo ip link set dev eth0 promisc off
```

### 6.10. Cambiar la MAC de una interfaz de red,

Tanto con ifconfig como con iproute2 hay que deshabilitar la tarjeta de red antes de cambiar la MAC y luego volver a activarla.

```
$ sudo ifconfig eth1 hw ether 08:00:27:75:2a:67  
$ sudo ip link set dev eth1 address 08:00:27:75:2a:67
```

### 6.11. Ver las entradas en la tabla ARP

```
arp -n ip neigh show
```

### 6.12. Añadir una entrada en la tabla ARP

**Comando antiguo.** `arp -i eth0 -s 192.168.0.1 00:11:22:33:44:55`

**Comando actual.** `ip neigh add 192.168.0.1 lladdr 00:11:22:33:44:55 nud permanent dev ens33`

En el caso de iproute2, tenemos que usar el objeto «neigh», tal y como podéis ver:

```
ip neigh add 192.168.0.1 lladdr 00:11:22:33:44:55 nud permanent dev ens33
```

### 6.13. Quitar una entrada de ARP

```
ip neigh del 192.168.0.1 lladdr 00:11:22:33:44:55 nud permanent dev ens33
```

Para ejecutar la ayuda de «ip neigh» y ver todos los comandos que podemos ejecutar, simplemente basta con poner:

```
ip neigh help
```

Tal y como habéis visto, las posibilidades de iproute2 con ip link, ip addr y ip neigh son casi infinitas, además, la sintaxis es muy intuitiva, pareciéndose muchísimo a los routers Cisco que tan conocidos son en el mundo de las redes.

### 6.14. Comandos «ip route» para configurar rutas

El comando «ip route» nos permitirá configurar la tabla de enrutamiento principal del sistema operativo, y también las tablas de enrutamiento adicionales que configuremos en el sistema operativo. A continuación, tenéis algunos ejemplos de uso con únicamente el comando ip route, sin utilizar el antiguo «route» que teníamos con la suite de «ifconfig».

### Ver la tabla de rutas

```
$ sudo ip route show
```

### Añadir una ruta en la tabla de enrutamiento

```
ip route add 10.8.0.0/24 via 192.168.1.2 dev eth0
```

### Borrar una ruta en la tabla de enrutamiento

```
ip route del 10.8.0.0/24 via 192.168.1.2 dev eth0
```

### Cambiar una ruta en la tabla de enrutamiento

```
ip route chg 10.8.0.0/24 via 192.168.1.3 dev eth0
```

### Agregar la puerta de enlace predeterminada (gateway)

```
ip route add default via 192.168.1.1 dev eth0
```

### Borrar la puerta de enlace predeterminada (gateway)

```
ip route del default via 192.168.1.254 dev eth0
```

### Ver el estado de la tabla de enrutamiento principal

```
ip route show ip route list
```

Tal y como podéis ver, agregar, borrar y cambiar rutas estáticas es realmente fácil, igual que definir una puerta de enlace predeterminada en nuestro sistema operativo.

## 6.15. Listado de comandos básicos de ss

ss se utiliza para volcar estadísticas sobre sockets. Permite mostrar información similar a netstat. ss muestra más información TCP y estado que otras herramientas. A continuación mostramos una serie de ejemplos con el comando ss y en algunos casos su equivalente con netstat.

### a) Ver estadísticas de sockets a la escucha

Con net-tools:

```
$ sudo netstat  
o también:  
$ sudo netstat -l
```

Con iproute2:

```
$ sudo ss  
o también:  
$ sudo ss -l
```

Ver estadísticas de sockets TCP a la escucha sin resolución de nombres (Volver al índice General)

```
$ sudo netstat -ltn
```

```
$ sudo ss -ltn
```

### b) Mostrar conexiones ssh establecidas

```
# ss -o state established '( dport = :ssh or sport = :ssh)'
```

| Recv-Q         | Send-Q       | Local Address:Port |
|----------------|--------------|--------------------|
| Peer           | Address:Port |                    |
| 0              | 0            | 192.168.1.36:60240 |
| 207.97.227.239 | ssh          |                    |

### c) Mostrar conexiones HTTP establecidas

```
ss -o state established '( dport = :http or sport = :http )'
```

### d) Hacer coincidir direcciones remotas y puertos

ss dst ADDRESS\_PATTERN

```
## Mostrar todos los puertos conectados de forma remota desde 192.168.1.5##  
ss dst 192.168.1.5
```

```
## Mostrar todos los puertos conectados de forma remota desde 192.168.1.5:http##  
ss dst 192.168.1.5:http  
ss dst 192.168.1.5:smtp  
ss dst 192.168.1.5:443
```

Mostrar conexiones realizadas por una ip en el puerto http hacia nuestra máquina

```
ss dst 123.1.2.100:http
```

Es posible mostrar todas las conexiones realizadas por cualquier máquina, por el puerto http por ejemplo:

```
# ss dst *:http
```

### e) Mostrar el número y el tipo de conexiones activas

```
ss -ant | awk '{print $NF}' | grep -v '[a-z]' | sort | uniq -c
```

### f) Listar todos los puertos a la escucha junto con el PID del programa

```
ss -tlnp
```

## g) Mostrar un resumen del estado de los sockets

```
# ss -s
```

### 6.16. Hacer persistentes todos los cambios en IPs y rutas

Si reiniciamos el servidor, todos los cambios que hayamos hecho en el sistema operativo con el comando «ip» no se guardarán al reiniciar el servidor, por tanto, debemos hacer persistentes estos cambios. Dependiendo del sistema operativo, se hace de una forma u otra. Lo primero que debemos hacer es editar el fichero de configuración ubicado en «/etc/network/interfaces» o en sistemas mas modernos que utilicen netplan */etc/netplan/\*.yaml*, en esos ficheros es donde podremos poner todas las direcciones IP y las rutas y reglas que nosotros deseemos.

## 7. El nombre del equipo - /etc/hostname

El fichero */etc/hostname* establece el nombre del ordenador local, el cual es leído durante el arranque del sistema. Debería contener una simple línea con el nombre de la máquina. Este nombre no puede superar los 64 caracteres de longitud, y se recomienda que esté formado por caracteres ASCII de 7 bits, en minúscula y sin espacios ni puntos.

## 8. Hacer un equipo que sea enrutador.

Una de las posibilidades que nos ofrece GNU/Linux es la de poder actuar como enrutador, es decir, recibir paquetes, decidir la ruta de estos y reenviarlos por cualquiera de las interfaces de red existentes. Para hacerlo necesitamos modificar el parámetro del kernel denominado *ip\_forward*.

9. El fichero de configuración */etc/sysctl.conf* se utiliza para ipestablecer algunos parámetros del kernel y que estos se mantengan entre sucesivos arranques del sistema, es decir, que los cambios sean persistentes. Esto es equivalente a cambiar valores en los archivos del directorio virtual */proc/sys*, sólo que con este último método los cambios se pierden al apagar el sistema.

El nombre completo del parámetro que debemos modificar para que el equipo funcione como router se denomina `net.ipv4.ip_forward`, y su valor debe ser 1. Para que este cambio se mantenga al reiniciar el sistema, editamos el fichero `/etc/sysctl.conf`, buscamos el parámetro, y cambiamos el valor cero por el valor uno:

```
net.ipv4.ip_forward = 1
```

Tras este cambio reiniciamos el sistema y el equipo ya funcionará a partir de ahora como router; pero podemos hacer el cambio en caliente y ahorrarnos el reinicio. Hay varias formas de hacer esto:

Directamente poniendo el 1 en el fichero del directorio `/proc/sys` que se encarga de este parámetro. La forma de descubrir el fichero relacionado con un parámetro concreto del fichero `/etc/sysctl.conf` es siempre la misma. Todos los ficheros están en `/proc/sys`, y los puntos del parámetro separan directorios hasta el último que separa el nombre del fichero, de esta manera, el fichero que buscamos es el `/proc/sys/net/ipv4/ip_forward`. Podemos escribir un 1 de la siguiente forma, sin necesidad de abrirlo con un editor:

```
# echo 1 > /proc/sys/net/ipv4/ip_forward
```

Con el comando `sysctl` podemos cambiar el valor de los parámetros del kernel de la siguiente manera:

```
# sysctl -w net.ipv4.ip_forward=1
```

El cambio anterior con `-w` no sería persistente.

También podemos cargar todos los parámetros del fichero `/etc/sysctl.conf` ejecutando:

```
# sysctl -p
```

Otros usos del comando `sysctl` son los siguientes:

Para ver todos los parámetros

```
# sysctl -a
```

Para ver sólo los parámetros que contengan una expresión regular:

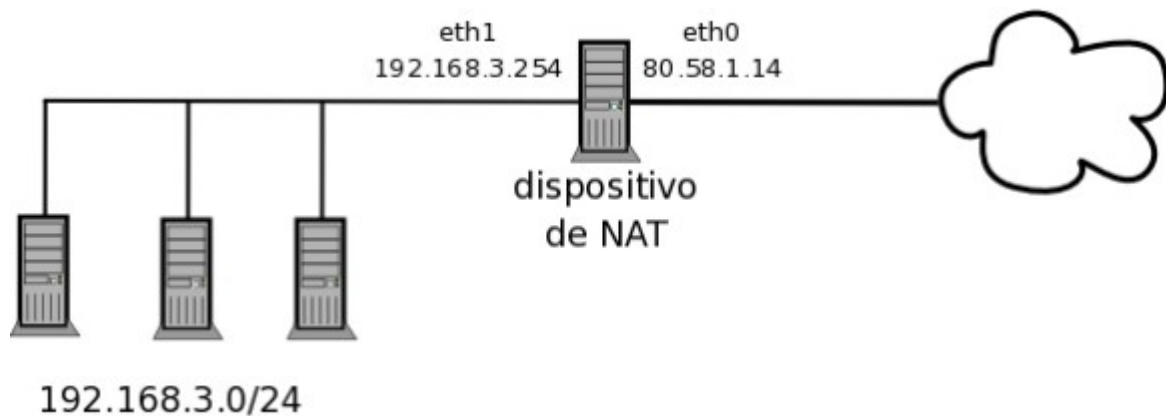
```
# sysctl -a --patter net.ipv4
```

Para ver un parámetro concreto:

```
# sysctl net.ipv4.ip_forward
```

## 10. Configuración NAT

Supongamos que tenemos una situación como la de la imagen:



Esquema de NAT

Vamos a ver los pasos que habría que dar para que todos los equipos de la red local tuviesen acceso a Internet y se pudiesen alojar servicios en cualquiera de ellos.

### 10.1. activación del bit de forward

Para que un equipo haga nat, primero tiene que enrutar.

#### 10.1. Postrouting

Todos los equipos de la red 192.168.3.0/24 están interconectados entre sí, pero en principio no tienen acceso a Internet puesto que sus direcciones IP son privadas y por tanto no son accesibles desde Internet (ningún equipo contestaría a sus peticiones). El

equipo que tiene dos interfaces de red sí tiene acceso a Internet ya que la interfaz de red eth0 tiene una dirección IP pública, además pertenece a la red 192.168.3.0/24 ya que está conectado a través de la interfaz de red eth1 con dirección IP 192.168.3.254.

El equipo con dos interfaces de red puede funcionar como dispositivo de NAT (source NAT), aceptando paquetes provenientes del resto de equipos de la red 192.168.3.0/24 que entren por eth1 con destino a cualquier equipo de Internet. Tal como se describe en el ejemplo inicial de SNAT, el dispositivo de NAT debe cambiar la dirección IP origen, pero esto se hace justo antes de enviar el paquete a Internet y por tanto habrá que definirlo en la cadena POSTROUTING.

## 10.2. Source nat (estático) con iptables

La regla que hay que poner para que se haga SNAT de todos los equipos de la red 192.168.3.0/24 es tan simple como:

```
iptables -t nat -A POSTROUTING -s 192.168.3.0/24 -o eth0 -j SNAT --to 80.58.1.14
```

Explicación de los parámetros:

- -A POSTROUTING: Añade (Add) una regla a la cadena POSTROUTING
- -s 192.168.3.0/24: Se aplica a los paquetes que tengan como dirección origen (source) la 192.168.3.0/24
- -o eth0: Se aplica a los paquetes que salgan (out-interface) por eth0
- -j SNAT --to 80.58.1.14 (--to aquí es equivalente a --to-source): Cambia la dirección de origen por la 80.58.1.14

Source nat (dinámico) con

Podríamos tener un caso similar al anterior, pero en el que la dirección IP pública del equipo que se conecta a Internet fuese dinámica, por lo que no la sabríamos a priori y no sería posible definirla en una regla como la anterior. En ese caso la regla de iptables a utilizar sería:

```
iptables -t nat -A POSTROUTING -s 192.168.3.0/24 -o eth0 -j MASQUERADE
```

Donde el único cambio se refiere a la acción (parámetro -j), en este caso es MASQUERADE, que cambia la dirección origen por la que tenga la interfaz de salida (eth0).

MASQUERADE podría funcionar también si la dirección IP de eth0 fuese estática, pero en ese caso se recomienda utilizar SNAT.

### 10.3. Prerouting

(Todo lo que se explica en este punto no se hace con la seguridad en mente, sino simplemente para explicar algunas cosas que se pueden hacer con PREROUTING. En una implementación real, esto tendría que ir combinado con un cortafuegos y un esquema de red diferente).

Para realizar una conexión cliente-servidor entre dos equipos de Internet hay que especificar completamente lo que se denomina socket de Internet, que queda definido con lo siguiente:

- Protocolo (normalmente TCP o UDP)
- Dirección IP equipo cliente
- Puerto equipo cliente
- Dirección IP equipo servidor

- Puerto equipo servidor

Si volvemos a nuestro problema, el único equipo de la red local que es accesible desde Internet es el dispositivo de NAT a través de su dirección IP pública 80.58.1.14, ya que sería el único con el que un equipo de Internet podría establecer un socket y por tanto sería el único equipo de la red que podría alojar servicios. Todo esto cambia si utilizamos NAT, ya que en el equipo que tiene las dos interfaces de red podemos cambiar la dirección IP destino (DNAT) de una petición que llegue de Internet y mandarla a un equipo de la red local.

Supongamos que instalamos un servidor web en un equipo de la red local con dirección IP 192.168.3.2 y queremos que sea accesible desde Internet, tendremos que modificar las peticiones que lleguen al puerto 80/tcp del equipo que tiene la dirección IP pública y que cambie la dirección IP destino 80.58.1.14 por 192.168.3.2, esto se hace con la siguiente regla:

```
iptables -t nat -A PREROUTING -p tcp --dport 80 -i eth0 -j DNAT --to 192.168.3.2
```

Explicación de los parámetros:

- -A PREROUTING: Añade (Add) una regla a la cadena PREROUTING
- -p tcp: Especifica el (p)rotocolo de transporte (tcp en este caso)
- --dport 80 (equivalente a --destination-port 80): Puerto destino 80 (ligada al parámetro anterior)
- -i eth0: Especifica eth0 como interfaz de entrada (in-interface)
- -j DNAT --to 192.168.3.2 (--to aquí es equivalente a --to-destination): Cambia la dirección IP destino (inicialmente 80.58.1.14) a 192.168.3.2

Es lógico que haya que hacerlo en la cadena PREROUTING, porque las reglas de esta cadena se aplican antes de tomar la decisión de enrutamiento, así se tomará la decisión de encaminamiento con la nueva dirección IP destino.

Para otros servicios bastaría con poner el protocolo y puerto adecuados, aunque el caso del servicio ftp es más complicado y necesitaría una discusión mas detallada. La principal limitación de utilizar DNAT con una sola dirección pública es que no es posible poner más de un servicio en el mismo puerto, ya que sólo se puede hacer DNAT a un equipo de la red local (el socket debe estar totalmente determinado).

Hay algunos servicios que permiten utilizar puertos diferentes a los estándar, como por ejemplo http, ya que podemos acceder a un servidor web que esté en un puerto diferente al 80/tcp, simplemente especificándolo en el navegador. Como iptables nos permite no sólo modificar la dirección IP destino sino también el puerto destino, podríamos poner un segundo servidor web en el equipo 192.168.3.3 y añadir la siguiente regla:

```
iptables -t nat -A PREROUTING -p tcp --dport 880 -i eth0 -j DNAT --to 192.168.3.3:80
```

Donde hemos especificado el puerto destino 880 (que no es un puerto estándar para ningún servicio) y cambiamos tanto la IP destino como el puerto destino con --to 192.168.3.3:80.

Para acceder a e

se servicio desde Internet tendremos que escribir en el navegador:

```
http://80.58.1.14:880
```

## 11. Guardar reglas iptables

### 11.1. Utilización del paquete iptables-persistent

Lo mas actual es utilizar el paquete `iptables-persistent` guardará las reglas que hayamos configurado

```
sudo apt-get install iptables-persistent
```

Para añadir nuevas reglas y queden persistentes se deben añadir al fichero `/etc/iptables/rules.v4`.

```
sudo nano /etc/iptables/rules.v4
```

Para asegurarnos de que los servicios estén habilitados en Debian o Ubuntu usando el comando `systemctl`:

```
sudo systemctl is-enabled netfilter-persistent.service
```

Si no, hay que habilitarlo:

```
sudo systemctl enable netfilter-persistent.service
```

Obtener estado:

```
sudo systemctl status netfilter-persistent.service
```

Para reiniciar el demonio y recargar las reglas iptables que están en el fichero `/etc/iptables /rules.v4`

```
sudo systemctl restart netfilter-persistent.service
```